ADS I Class Project **Summer 2026**
Hardware Design with Chisel
Prof. Dr.-Ing. Wolfgang Kunz RPTU Kaiserslautern-Landau
M.Sc. Tobias Jauch Fachbereich Elektrotechnik und Informationstechnik
Andro Mazmishvili Entwurf informationstechnischer Systeme
Juriaan Muijsson
Tristan Schmitt

Assignment 4 Hazard Detection and Forwarding Unit

The fourth assignment extends the pipelined processor from task 3 with the capabilities to detect and resolve hazards. This functionalitiy is implemented in a forwarding unit.

Structure of the Project

This task uses your implementation from task 3 as a starting point. Make sure that your pipelined core is working correctly before starting with this assignment.

Task 4.1 Forwarding Unit

In a pipelined processor, subsequent instructions might have data dependencies. For instance, one instruction writes to a register that the following instruction reads from. In your design from task 3, without a way to detect and resolve such a hazard, the core would produce a wrong result. A forwarding unit detects and resolves data hazards in pipelined processor cores by forwarding values from later pipeline stages to the execute stage, allowing instructions to continue without stalling.

a) Preparation Before you start with the implementation, please answer the following questions:

1. What signals are required as inputs / outputs to the forwarding unit?
2. What types of hazards can be solved by forwarding?
3. Which pipeline stages are connected to the forwarding unit, and how can potential hazards be detected?
4. Take a look at ADS I slide 6-24. What does it tell you about simultaneous read and write requests to the register file? How can the desired behavior be implemented in the register file from task 3?
5. Are there any other potential hazards left in your specific implementation that would require stalling?

b) Implementation Add a forwarding unit to your processor design from task 3, and stick to the design presented in the lecture (slide 6-24ff, schematic on slide 6-26)

c) Test your Implementation Check whether your design runs and produces correct results by adapting the test cases you created in the previous assignments to include all types of dependencies that create data hazards and all possible forwarding scenarios. The test bench is located in PipelinedRISCV32I_tb.scala, test cases need to be added in HEX assembly to the "Binary File" in the "programs" folder.
A RISC-V encoder and a RISC-V interpreter might be helpful for you to achieve this task.